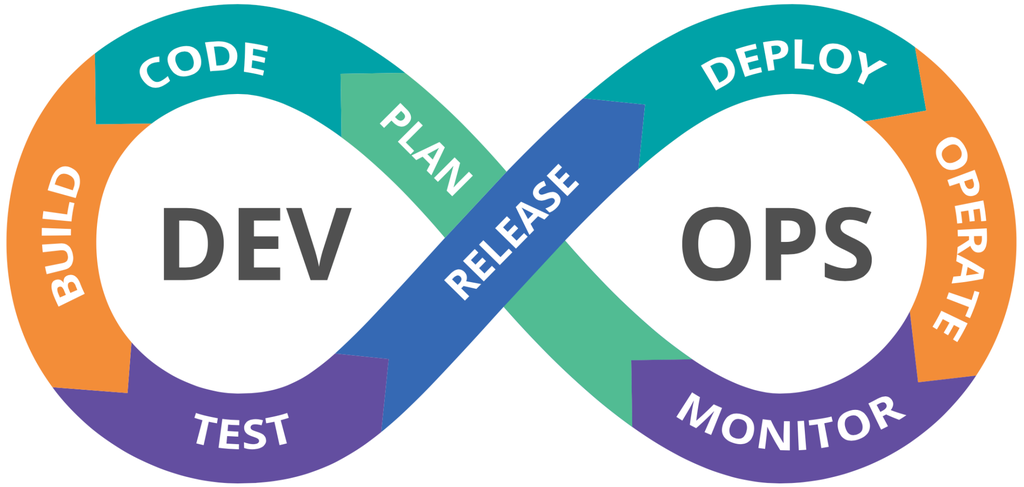
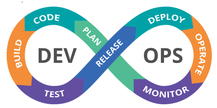


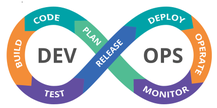
Nastanak i tri osnovna principa

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



DevOps - nastanak

- Nekoliko različitih pravaca u upravljanju poslovanjem kao i različitih tehnoloških pristupa su postavili osnov za nastanak DevOps-a
- Za razumevanje DevOps-a, neophodno je razumeti koncept tokova stvaranja nove vrednosti (engl. *value streams*)
- DevOps predstavlja primenu principa racionalne proizvodnje (*Lean practice*) u sferi IT-a

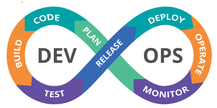


DevOps - 3 principa

- **Princip toka** (engl. *Flow*) - ubrazava isporuku rezultata rada (razvoja) od razvojnog tima ka operativnom timu i klijentu

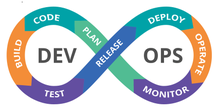
- **Princip povratnih informacija** (engl. *Feedback*) - ustanovljavanje ovog principa omogućava povećanje bezbednosti i kvaliteta rada

- **Princip kontinualnog učenja i eksperimentisanja** - promoviše kulturu visokog poverenja i naučnih principa pri preuzimanju rizika u svakodnevnim poslovima



Pokreti koji su uticali na razvoj DevOps principa

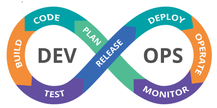
- **Pokret racionalnih (proizvodnih) praksi** - promovisala ga Toyota - na kraju je prizvodni sistem i kodifikovan - mapiranje tokova stvaranja nove vrednosti, Kanban table, Total Productive Maintanance. Ove prakse se fokusiraju na to kako kreirati novu vrednost za klijenta (koji je onda i zadovoljan kupac), putem sistematskog razmišljanja o sledećim stvarima: - svrha onoga što se radi, naučno razmišljanje (pristup problemu), kreiranje tokova (resursa), obezbeđivanje kvaliteta na samom izvoru, upravljanje na uljudan način, vodeći računa o pojedincima uključenim u procese



Pokreti koji su uticali na razvoj DevOps principa

- **Agilni manifest** - ideja da se po ugledu na racionalne proizvodne prakse stvori “lagan” (lightweight) skup preporuka (nasuprot komplikovanim procesima razvoja softvera). Ključni princip je “isporučiti funkcionalan sotver često, u intervalima od nekoliko nedelja do nekoliko meseci, pri čemu je poželjno da taj period bude kraći”. Naglasak je na što češćim isporukama malih inkrementalnih novih funkcionalnosti.

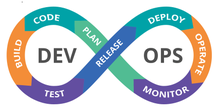
  - ovaj pokret je po mnogima zaslužan za drastično unapređenje produktivnosti mnogih softverskih kompanija



Pokreti koji su uticali na razvoj DevOps principa

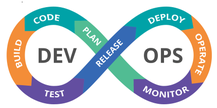
- **Agilna infrastruktura i pokret “Velocity”** - 2008. je predloženo da se agilni principi primene i na samu infrastrukturu.

- Na Velocity konferenciji 2009, John Allspaw i Paul Hammond prezentuju rad “10 Deploys per Day: Dev and Ops Cooperation at Flickr”, gde su opisali uspostavljanje zajedničkih ciljeva za Dev i Ops timove, i upotrebu principa kontinualne integracije, kako bi posao instaliranja gotovog proizvoda (*deploymenta*) postao deo svakodnevne rutine svih članova tima.



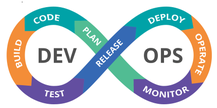
Pokreti koji su uticali na razvoj DevOps principa

- **Pokret kontinualne isporuke** - nadograđujući koncepte kontinualnog bildovanja, testiranja i integracije Jez Humble i David Farley su osmislili kontinualnu isporuku (*continuous delivery*) - gde se definiše i uloga ***deployment pipeline***-a, koji treba da obezbedi da su infrastruktura i sam kod aplikacija uvek u stanju da se isporuče klijentu.



Pokreti koji su uticali na razvoj DevOps principa

- **Toyota Kata** - iako su mnoge firme usvojile racionalne principe proizvodnje rezultat koji je postigla Toyota je izmicao - nedostajao ključni deo - *improvement kata.* Uspostavljanje filozofije konstantog unapređenja dnevnih rutina - stalni ciklus prepoznavanja novih stanja koja se žele dostići, postavljanje sedmičnih ciljeva, i konstanto unepređenje svakodnevnih aktivnosti.



Šta je *value stream*?

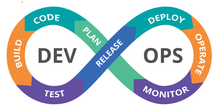
- “Sled aktivnosti koju određena organizacija sprovodi kako bi ispunila zahteve korisnika”.

- “Sled aktivnosti koje su neophodne da se izvrši dizajn, proizvodnja i isporuka dobara i usluga krajnjem korisniku, ubuhvatajući pri tome dvosmerni tok informacija i materijala”.

Value Stream Mapping: How to Visualize Work and Align Leadership for Organizational Transformation

Karen Martin & Mike Osterling

- U proizvodnim delatnostima ovaj tok stvaranja nove vrednosti je lako identifikovati - započinje narudžbom kupca i isporukom materijala od koga počinje proizvodnja



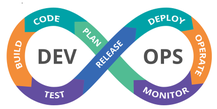
Technology value stream?

- Isti principi i šabloni rada se mogu primeniti i na tehnološke procese prisutne u IT sektoru
- DevOps tehnološki tok stvaranja nove vrednosti posmatra kao proces koji je neophodan da se poslovni zahtevi (hipoteza) transformišu u tehnološki podržane servise koji korisniku obezbeđuju neku novu višu vrednost.
  - Polazna tačka procesa je poslovni cilj, osmišljeni koncept, ideja, a započinje u onom momentu kada se prihvati razvoj (rad na takvom projektu) u razvojnom timu.
  - Razvojni timovi (koji rade agilnim, iterativnim postupkom) transformišu ove ideje u User Story, pa zatim na osnovu toga u specifikaciju zahteva, koji se zatim implementiraju programiranjem i dalje kroz lanac - kod se isporučuje na sistem za verzioniranje (npr. Git), integriše, testira i pakuje za isporuku.
- **Nova vrednost za korisnika nastaje samo kada je kod uspešno isporučen u produkciji, pa je važno da se verzije isporuče brzo, ali i pouzdano, bez izazivanja zastoja i problema u produkciji.**



Mere koje ukazuju na efikasnost organizacije

- Vreme isporuke - **Deployment Lead Time**
- Procesno vreme - **Process Time**
- **%C / A mera**

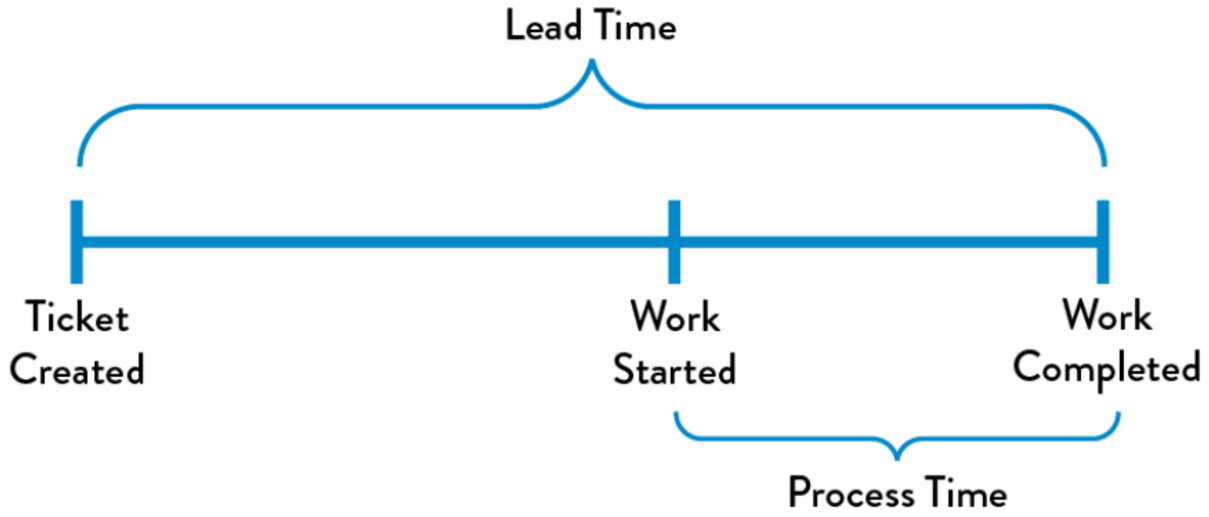


Deployment lead time

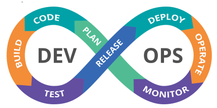
- Vreme isporuke - vremenski interval koji protekne od momenta kada neki zahtev bude prihvaćen za realizaciju do momenta kada je on uspešno primenjen u produkciji.
- Skraćenje ovog vremena je u fokusu većine praksi DevOps-a
  - Početna faza - Dizajn i razvoj (engl. *Design and Development*) zahteva visok nivo kreativnosti i dosta uloženog, često jedinstveno utrošenog rada (koji neće biti ponavljan). Ovde je utrošeno vreme često vrlo varijabilno.
  - Druga faza - Testiranje i operativni nadzor (engl. *Testing and Operations*) - takođe zahteva kreativnost i ekspertizu, ali je predvidljivog vremena trajanja, a sam cilj je da se postignu zadovoljavajući rezultati sa minimizacijom varijabilnosti.
- Kako bi se postigli najbolji efekti cilj je da se radni proces izvršava nad manjim radnim celinama (nasuprot *Waterfall* pristupu ili jako velikim dugotrajnim granama razvoja velikih *feature-a*) čime se omogućava simultani rad prve i druge faze.

Lead Time / Process Time

- Procesno vreme je vremenski interval koji protekne od momenta kada se započne razvoj nekog prihvaćenog zahteva do momenta kada je on uspešno u produkciji.



slika preuzeta iz knjige “DevOps Handbook”,
Gene Kim, Jez Humble, Patrick Debois, John Willis

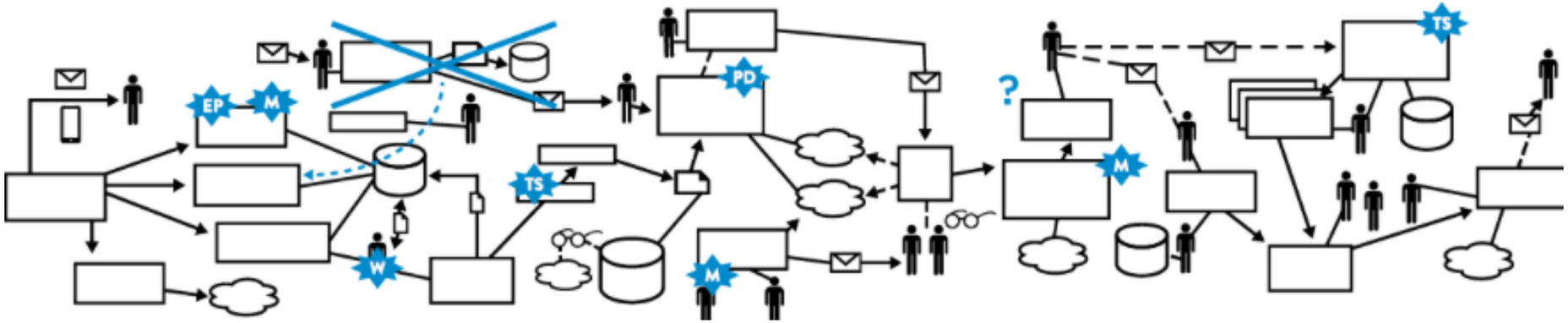


Lead Time / Process Time

- Fokus je obično na optimizaciji celokupnog *Lead Time*-a (jer je to ono što vidi krajnji korisnik)
- Ipak, odnos procesnog vremena i vremena isporuke je važna mera (indikator) efikasnosti same organizacije
  - Kako bi se postigao brz protok radnih zadataka neophodno je imati što manje redove čekanja - brz razvoj ne vredi mnogo ako je količina nagomilanih poslova koja čeka realizaciju ogromna

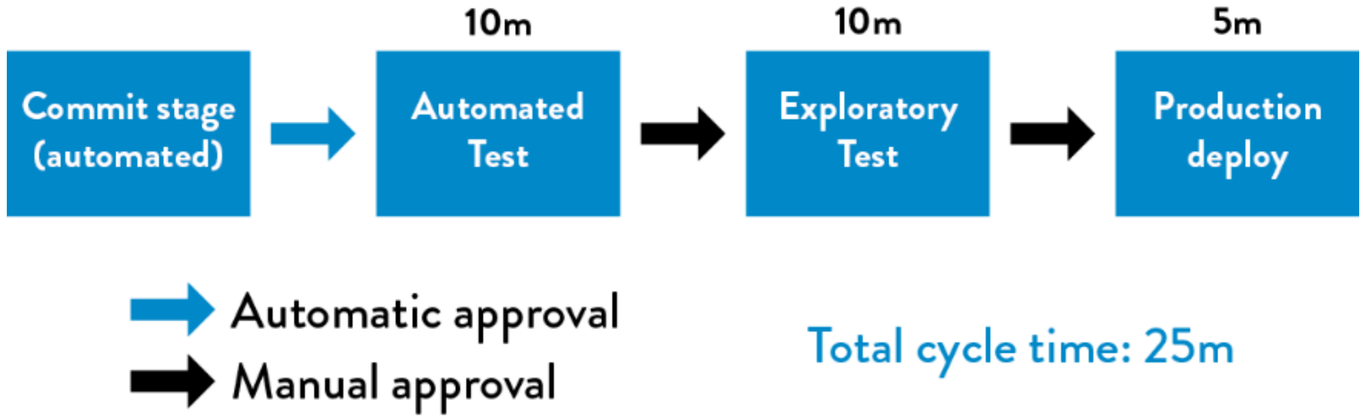
Klasični scenario - vreme isporuke nekoliko meseci

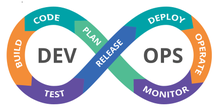
- Vreme isporuke od nekoliko meseci je tipično za velike i složene organizacije koje rade na razvoju jako spregnute, monolitne aplikacije, sa slabo integrisanim i dugotrajnim testiranjem i velikim stepenom manuelnog testiranja.
- U ovakvim situacijama *value stream* je zamršen i komplikovan, i neophodan je “herojski” napor pojedinaca u svakom koraku da bi se razvoj uspešno priveo kraju.



DevOps idealni scenario - vreme isporuke nekoliko minuta

- U ovakvom scenariju razvojni tim brzo i konstantno dobija povratne informacije o rezultatima primene u produkciji, koristeći te informacije za brzu adaptaciju - brzu primenu, integraciju i validaciju koda i isporuku izmenjene verzije u produkciono okruženje.
- Ovo se postiže konstantnom proverom svih malih izmena u kodu, automatskim testiranjem i isporukom izmena u malim “paketima”. Na ovaj način se postiže sigurnost da izmene rade kako je predviđeno i ne unose poremećaj.
- Arhitektura projektovanog sistema je modularna, dobro enkapsulirana i slabo povezana.





% C / A

- Procenat kompletno i tačno realizovanih zahteva je treća mera koja ukazuje na efikasnost organizacije
- Može se utvrditi tako što se anketiraju krajnji korisnici koji procenat onoga što im je isporučeno mogu odmah, bez ikakvih izmena i prilagođavanja da koriste u datom stanju - bez potrebe da se ponovo pojašnjava zahtev koji su imali

Tri principa

- Principi koji su u osnovi svih DevOps praksi i obrazaca
  - Princip toka
  - Princip brzih povratnih informacija iz produkcije ka razvojnom timu
  - Princip kreiranja visokopouzdanog razvojnog okruženja sa kontinualnim povratnim informacijama

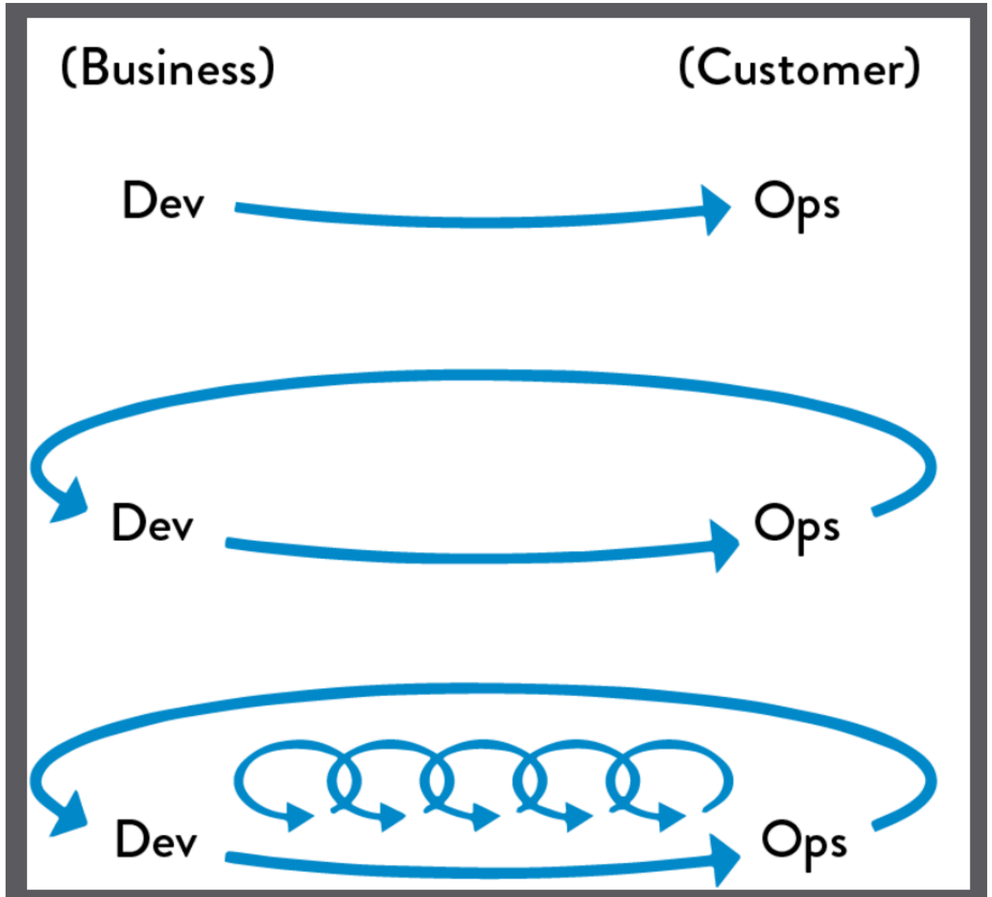
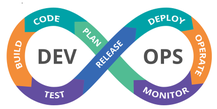
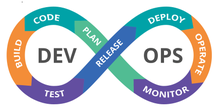


Figure 5: *The Three Ways (Source: Gene Kim, "The Three Ways: The Principles Underpinning DevOps," IT Revolution Press blog, accessed August 9, 2016, http://itrevolution.com/the-three-ways-principles-underpinning-devops/.)*



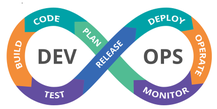
Princip toka

- Obezbeđuje brz protok informacija i rada od razvojnog ka operativnom timu i krajnjem klijentu
- Kako bi se maksimizovao protok - posao mora biti vidljiv, zadaci redukovani na male celine i kratke intervale razvoja, razvoj mora od početka voditi računa o kvalitetu (čime se sprečava propagacija greške kroz ostatak lanca)
- Smanjuje se vreme isporuke i podiže kvalitet
- Rezultujuće prakse koje podržavaju ovaj princip
  - kontinualno bildovanje, integracija i testiranje, proces isporuke (deployment process), kreiranje infrastrukturnih okruženjana zahtev



Princip brzih povratnih informacija

- Zahteva da se “pojača” kanal za prikupljanje povratnih informacija - s ciljem da se pravovremenim upoznavanjem sa realnim stanjem aplikacija u produkciji izbegne ponavljanje grešaka koje utiču na krajnjeg korisnika, kao i da se obezbedi okruženje koje omogućava brzu detekciju i oporavak od grešaka.
- Na ovaj način obezbeđuje se razvoj kvalitetnog koda na samom izvoru, kao i prenos potrebnog znanja do članova tima koji tako postaju svesni činjenica koje utiču na sam razvoj



Princip kreiranja visokopouzdanog razvojnog okruženja sa kontinuiranim tokovima povratnih informacija

- Formira se viskopouzdano okruženje za razvoj i isporuku softvera
- dinamičko, disciplinovano okruženje, koje primenjuje i naučne principe eksperimentisanja i procene rizika, okruženje koje uči iz uspešnih i neuspešnih pokušaja
- Ovde se dodatno pojačava princip prikupljanja povratnih informacija, tako što se formira veliki broj malih povratnih petlji u svim fazama razvoja.